

Trabajo Práctico 3 — Visión Computacional con CNNs

Clasificación de imágenes satelitales EuroSAT (PyTorch)

Juan Ignacio Elosegui Juan González Merlhe
Tecnología Digital VI — Inteligencia Artificial

Junio 2026

1. CNN propia entrenada desde cero

Arquitectura. Diseñamos una CNN de 4 bloques convolucionales, cada uno con el patrón `Conv(3 × 3) → BatchNorm → ReLU → MaxPool`, que duplica los canales ($3 \rightarrow 32 \rightarrow 64 \rightarrow 128 \rightarrow 256$) mientras reduce la resolución espacial de 64×64 a 4×4 . En lugar de aplanar el mapa de features usamos *Global Average Pooling*, seguido de una cabeza densa ($256 \rightarrow 128 \rightarrow 10$) con `Dropout`. El modelo tiene apenas **423.562 parámetros**, muy compacto. `BatchNorm` y `Dropout` actúan como regularizadores y estabilizan el entrenamiento.

Pipeline de datos y data augmentation. Cargamos los datos con `Dataset/DataLoader`. Sobre el conjunto de entrenamiento aplicamos aumentaciones acordes a la naturaleza satelital de las imágenes, donde no existe una orientación privilegiada: *flips* horizontal y vertical, rotaciones de hasta 90° y un `ColorJitter` suave de brillo/contraste/saturación. Estas transformaciones explotan las invarianzas geométricas del dominio y reducen el sobreajuste. Validación y test usan únicamente normalización (evaluación determinística).

Entrenamiento y monitoreo. Entrenamos 30 épocas con Adam ($\text{lr} = 10^{-3}$, $\text{weight_decay} = 10^{-4}$) y `ReduceLROnPlateau`, optimizando *cross-entropy*. La Figura 1 muestra la evolución de *loss* y *accuracy* en train y validación: la curva de validación es ruidosa pero con tendencia creciente hasta estabilizarse en torno al 0,94 de *accuracy*.

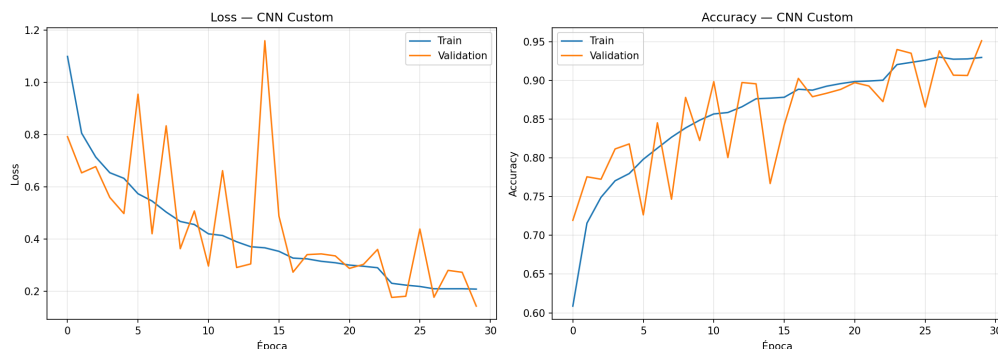


Figura 1: Evolución de *loss* y *accuracy* (train vs. validación) de la CNN propia.

Flujo de gradientes. Para descartar *vanishing/exploding gradients* registramos la norma del gradiente por capa durante las primeras épocas (Figura 2). Las normas se mantienen acotadas en un rango sano ($\sim 10^{-1}$ a $\sim 10^1$), sin colapsar a cero ni dispararse: `BatchNorm` cumple su rol y el flujo de gradientes es estable en toda la red.

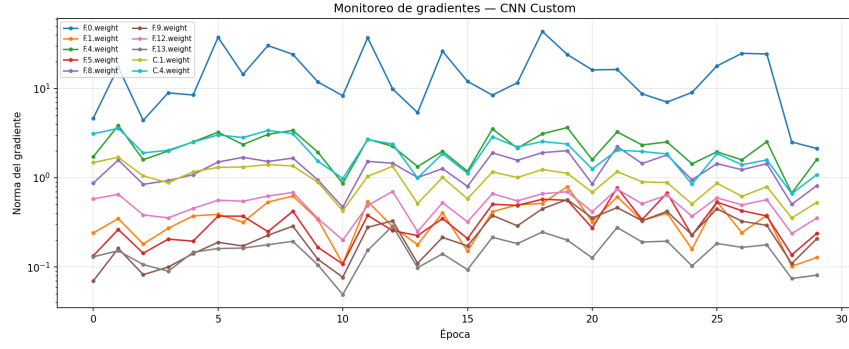
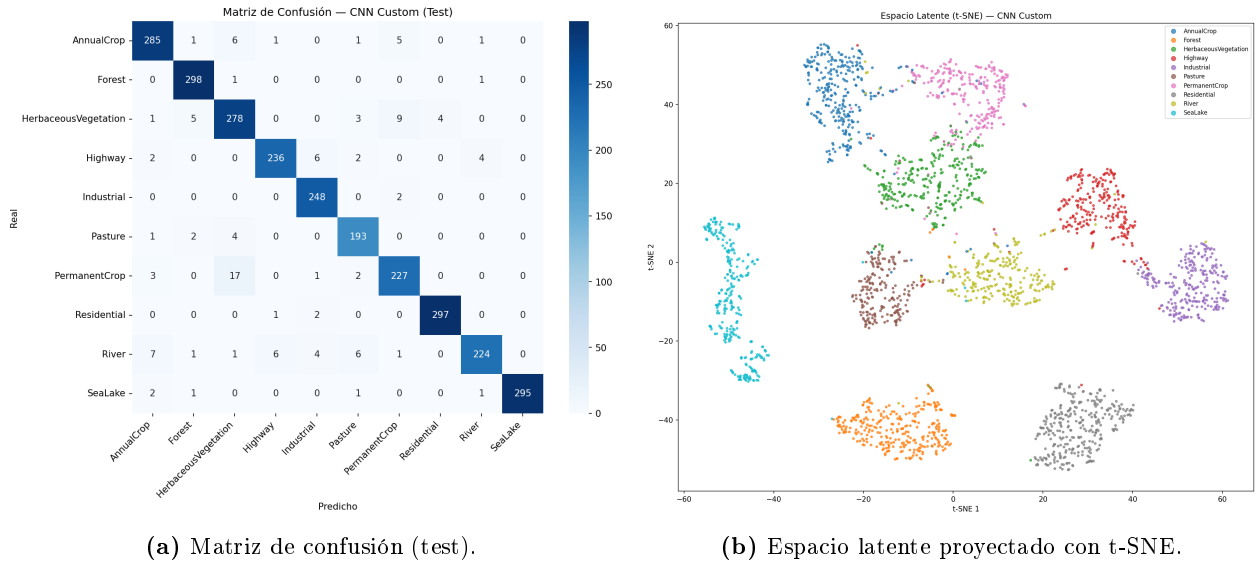


Figura 2: Norma del gradiente por capa (escala log). No se observa desvanecimiento ni explosión.

2. Análisis de errores y espacio latente

Sobre el conjunto de **test** la CNN propia alcanza un **accuracy del 95,6 %** (2581/2700). La matriz de confusión (Figura 3a) muestra que el grueso del error se concentra entre clases agrícolas/vegetación, visualmente muy parecidas desde un satélite: *PermanentCrop*→*HerbaceousVegetation* (17 casos), el sentido inverso (9) y *River*→*AnnualCrop*/*Highway* (cuerpos de agua angostos confundidos con caminos o parcelas). Clases con textura distintiva (*Forest*, *Residential*, *SeaLake*) son casi perfectas. Estas confusiones reflejan la ambigüedad inherente del uso de suelo, donde las fronteras entre categorías son difusas.



(a) Matriz de confusión (test).

(b) Espacio latente proyectado con t-SNE.

Figura 3: Análisis de la CNN propia: errores por clase y estructura del espacio latente.

Espacio latente. Extrajimos los *features* de 128 dimensiones previos a la capa de clasificación y los proyectamos a 2D con t-SNE (Figura 3b). La mayoría de las clases forman *clusters* distinguibles *antes* de la decisión final, lo que indica que la red aprendió representaciones discriminativas. Los *clusters* que se solapan (cultivos/vegetación) coinciden con las confusiones de la matriz, confirmando que el error proviene de clases genuinamente similares y no de un fallo de aprendizaje.

3. Transfer Learning con ResNet18

Partimos de una **ResNet18** preentrenada en ImageNet. Aplicamos *fine-tuning* selectivo: congelamos las capas iniciales y descongelamos solo **layer4** y reemplazamos la capa final (**fc**) por una nueva de

10 salidas con **Dropout** ($\sim 8,4\text{M}$ parámetros entrenables de $\sim 11,2\text{M}$ totales). Como ResNet espera entradas de 224×224 , redimensionamos las imágenes. Entrenamos 15 épocas con Adam ($1r = 10^{-4}$).

Métrica	CNN propia	ResNet18 (TL)
Accuracy en test	95,6 %	98,0 %
Mejor accuracy en validación	$\sim 0,94$	$\sim 0,977$
Épocas para converger	30 (ruidosa)	~ 3 (estable)
Parámetros totales	0,42 M	11,2 M
Parámetros entrenables	0,42 M	$\sim 8,4$ M
Costo de cómputo / época	bajo (64^2)	alto (224^2 , $\sim 6\times$)

Cuadro 1: Comparación entre la CNN propia y el *transfer learning* con ResNet18.

La Figura 4 compara la convergencia de ambos modelos. ResNet18 **parte ya en $\sim 0,95$** de *accuracy* en la primera época y se estabiliza casi de inmediato, mientras que la CNN propia necesita decenas de épocas ruidosas para acercarse. Esto se debe a que las features de bajo nivel aprendidas en ImageNet (bordes, texturas, formas) son transferibles al dominio satelital.

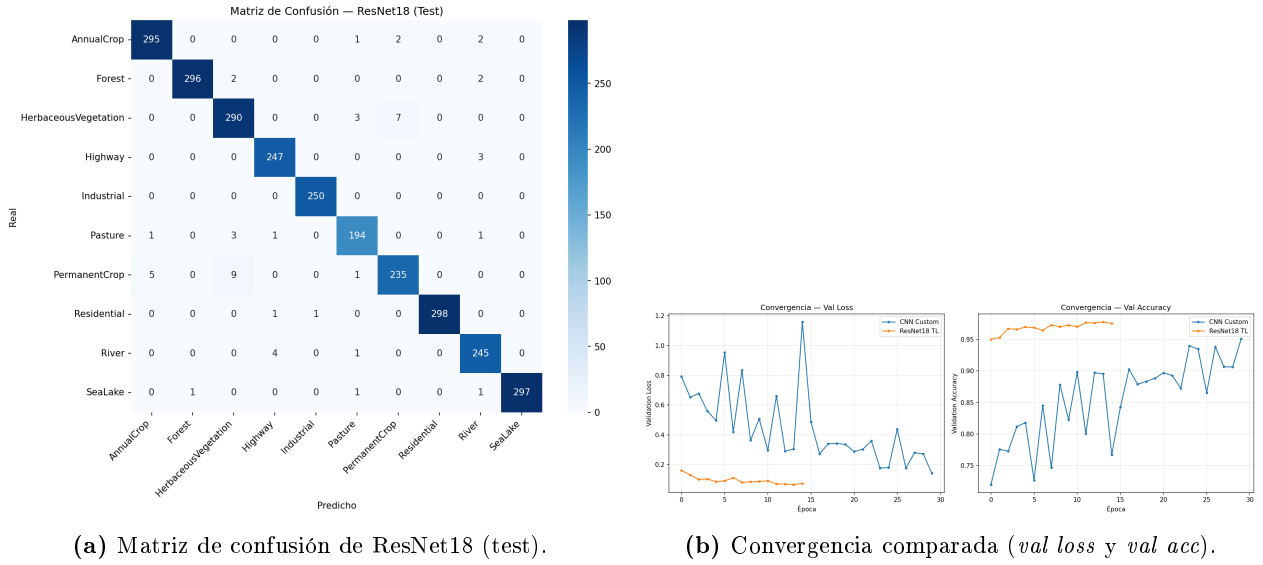


Figura 4: Resultados de ResNet18 y comparación de convergencia con la CNN propia.

ResNet18 no solo mejora el *accuracy* global sino que **reduce a la mitad** las confusiones más difíciles (p.ej. *PermanentCrop* $\rightarrow$ *HerbaceousVegetation* baja de 17 a 9 casos), aunque el solapamiento cultivos/vegetación persiste por ser intrínsecamente ambiguo.

4. Conclusiones

- La CNN propia, con apenas 0,42M de parámetros, logra un sólido 95,6 % de *accuracy* y un espacio latente bien estructurado, validando el diseño.
- El *transfer learning* con ResNet18 es superior en todos los ejes salvo el costo por época: converge en pocas épocas y alcanza 98,0 %, gracias a las features preentrenadas.
- El mayor costo por época de ResNet18 (entradas 224×224 y más parámetros) se compensa con la drástica reducción en la cantidad de épocas necesarias.
- El *fine-tuning* de solo **layer4** + **fc** es efectivo: conserva las features generales de bajo nivel y adapta las de alto nivel al dominio satelital.